WAFT Kernel Implementation

ABSTRACT

Instead of creating new systems, we integrated with existing infrastructure.

1. Introduction

Instead of creating new systems, we integrated with existing infrastructure. We use TheObserver for flight recorder logging, extend the existing EvolutionaryEventType enum, and integrate with EmpiricaManager for epistemic state. This approach avoids duplication and leverages proven systems.

2. Architecture

> ### THE FLIGHT RECORDER
>
> Instead of creating new systems, we integrated with existing infrastructure. We use TheObserver for flight recorder logging, extend the existing EvolutionaryEventType enum, and integrate with EmpiricaManager for epistemic state. This approach avoids duplication and leverages proven systems.

3. Methodology

We implemented the complete WAFT Kernel Boot Sequence, which enables the kernel to acknowledge its identity, perform self-awareness checks, and integrate with the existing status system. The kernel is the central operating intelligence that oversees directed evolution of self-modifying AI agents. We extended the EvolutionaryEventType enum to include BOOT and STATUS_CHECK event types. This allows the kernel to log its lifecycle events to the flight recorder using the existing TheObserver system. We created a new kernel module with an epistemic phase calculator that determines the current phase from Empirica state: Data Gathering, Exploration, Synthesis, Evolution, or Transition. We significantly enhanced the status script with kernel awareness. The status check now includes epistemic state from Empirica, showing knowledge percentage, uncertainty percentage, coverage, and the current epistemic phase. We

added comprehensive path validation throughout to prevent security vulnerabilities, and all operations now handle missing components gracefully.

4. Conclusion

We added path validation on all file operations to prevent path traversal attacks. Work effort directory names are validated, git file paths are checked, and _pyrite paths are verified. All inputs are validated throughout the system. We created a new boot command handler that executes the complete boot sequence: identity acknowledgment, initial status check, epistemic phase declaration, and boot event logging. The command documentation provides clear instructions for using the kernel. The implementation is complete and ready for testing. All components integrate properly with existing systems. The kernel can now acknowledge its identity, perform status checks with epistemic awareness, and log events to the flight recorder. The system handles errors gracefully and validates all paths for security.